COM3502/4502/6502 SPEECH PROCESSING

Lecture 08

The Fourier Transform

Fourier Transforms (Definition)

The Fourier Transform (FT) transforms continuous time signals to the continuous frequency domain.

$$\mathrm{FT}\{x(t)\} = \mathrm{x}(j\omega) = \int_{-\infty}^{\infty} x(t)e^{-j\omega t}dt$$

$$\mathrm{IFT}\{\mathrm{x}(j\omega)\} = x(t) = \frac{1}{2\pi}\int_{-\infty}^{\infty} \mathrm{x}(j\omega)e^{j\omega t}d\omega$$

The Discrete Time Fourier Transform (DTFT) transforms discrete (sampled) time signals to the continuous frequency domain.

$$\mathrm{DTFT}\{x[k]\} = \mathrm{x}(e^{j\Omega}) = \sum_{k=-\infty}^{\infty} x[k]e^{-j\Omega k}$$

$$\mathrm{IDTFT}\{\mathrm{x}(e^{j\Omega})\} = x[k] = \int_{-\pi}^{\pi} \mathrm{x}(e^{j\Omega})e^{j\Omega k}d\Omega$$

The Discrete Fourier Transform (DFT) transforms discrete time signals to the discrete frequency domain. (IDFT: Inverse DFT)

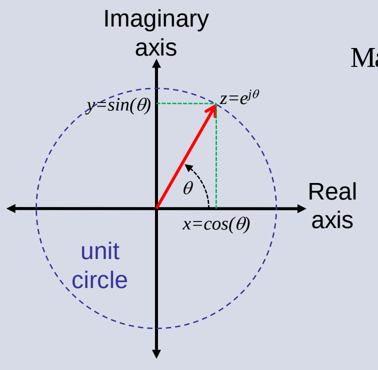
(FFT is no different transform but an efficient algorithm for the DFT!)

$$\mathrm{DFT}\{x[k]\} = \mathrm{x}[n] = \sum_{k=0}^{L_{DFT}-1} x[k]W_{L_{DFT}}^{kn} \quad \text{with } W_{L_{DFT}} = e^{j2\pi/L_{DFT}}$$

$$\mathrm{IDFT}\{\mathrm{x}[n]\} = x[k] = \sum_{n=0}^{L_{DFT}-1} \mathrm{x}[n]W_{L_{DFT}}^{-kn}$$

Complex Numbers: *A Reminder*

$$z = x + jy$$



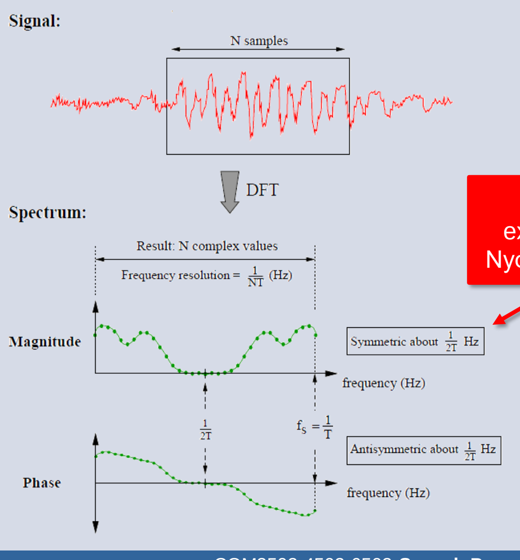
$$\text{Magnitude } |z| = \sqrt{x^2 + y^2}$$

$$\text{Phase } \theta = \tan^{-1}\left(\frac{y}{x}\right)$$

$$e^{j\theta} = \cos(\theta) + j\sin(\theta)$$

15

Using the DFT



This is the explanation for Nyquist's theorem

N: DFT length

T: sampling interval

f_s: sampling frequency

18

Implicit Periodicity in the DFT

- The DFT computes the spectrum at N evenly spaced discrete frequencies
- The algorithm assumes periodicity <u>outside</u> the analysis frame (*with a period equal to the frame length*)
- This means that <u>discontinuities</u> will arise for …
  - periodic signals with a *non-integer* number of cycles in the analysis frame
  - <u>all</u> aperiodic signals
  - <u>all</u> stochastic signals
- Such discontinuities give rise to unwanted spectral components

University of Sheffield

FT vs. DFT

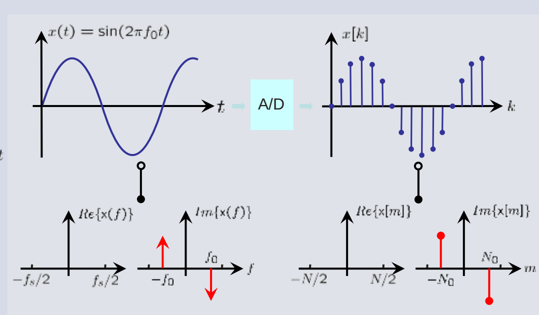
continuous

time domain.

$$x(j\omega) = \int_{-\infty}^{\infty} x(t) e^{-j\omega t} dt$$

continuous

freq. domain.



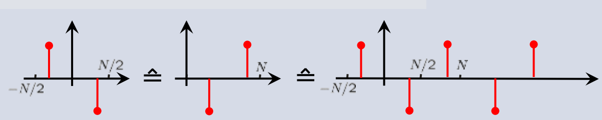
discrete

time domain.

$$x[n] = \sum_{k=0}^{N-1} x[k] e^{j2kn\pi/N}$$

discrete

freq. domain.

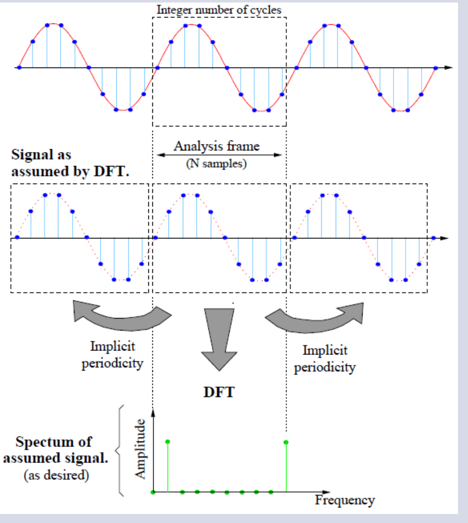


In (e.g.) MATLAB / Numpy: first pos. freqs., then neg. frequencies

21

University of Sheffield

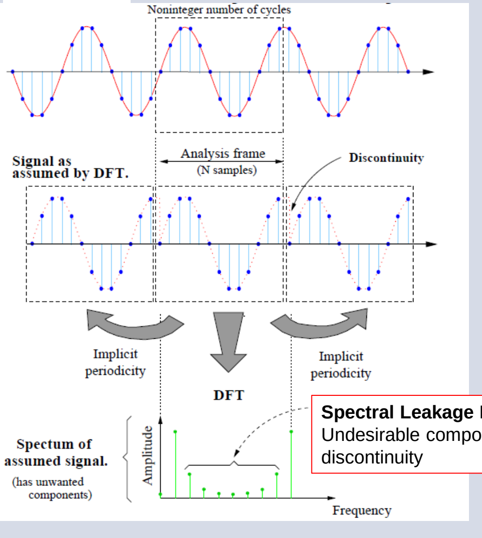
Implicit Periodicity in the DFT



University of Sheffield

© The University of Sheffield

Implicit Periodicity in the DFT



Spectral Leakage Effect
Undesirable components due to discontinuity

University of Sheffield

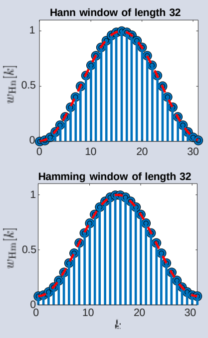
Windowing

- The discontinuities arising from segmenting the signal into frames distorts the spectrum
- The distortion can be reduced by multiplying each signal frame with a 'window function'
- The most common window functions are
  - the 'Hann (Hanning) Window' (raised cosine window, *proposed by Julius von Hann*)

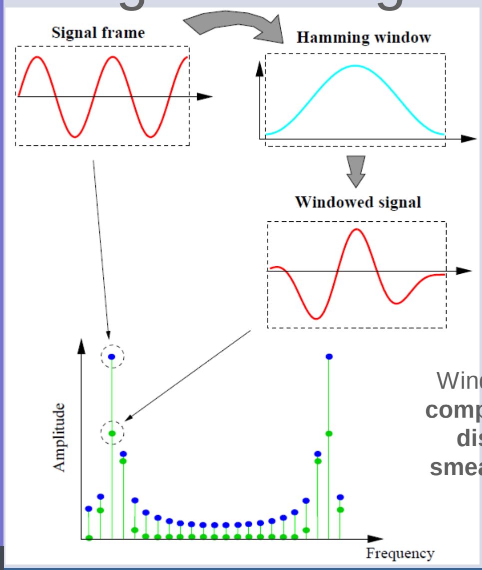
    $$w_{\mathrm{Hn}}[k] = 0.5 \cdot \left(1 - \cos\left(\frac{2 \cdot \pi \cdot k}{L}\right)\right)$$

  - the 'Hamming Window' (*proposed by Richard W. Hamming*) …

    $$w_{\mathrm{Hm}}[k] = 0.54 - 0.46 \cdot \cos\left(\frac{2 \cdot \pi \cdot k}{L-1}\right)$$

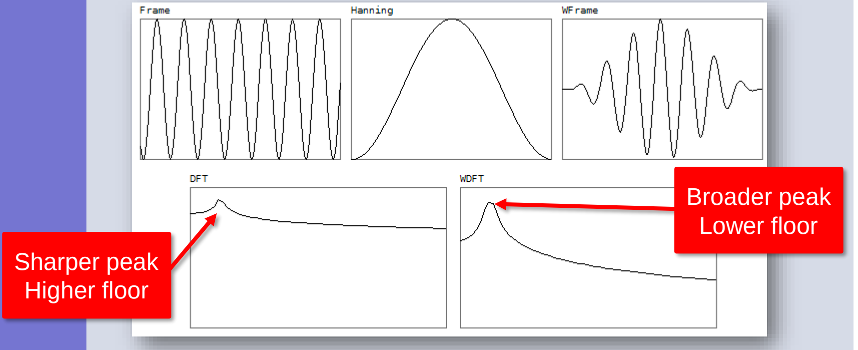


Windowing – Leakage Reduction



Windowing **attenuates the components caused by the discontinuity**, but also **smears the spectral peaks**

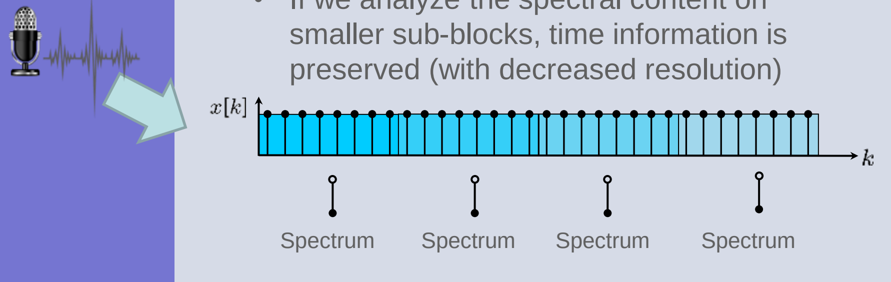
Windowing



28

Block-wise Frequency-Analysis

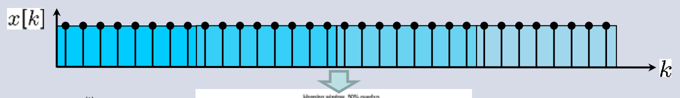
- If we analyze the spectral content on smaller sub-blocks, time information is preserved (with decreased resolution)

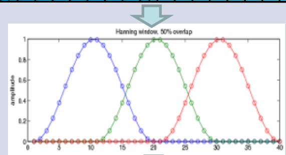


29

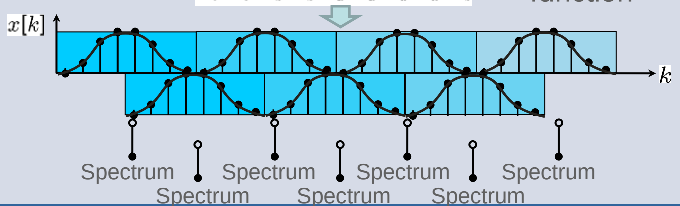
Block-wise Frequency-Analysis

- If we analyze the spectral content on smaller sub-blocks, time information is preserved (with decreased resolution)

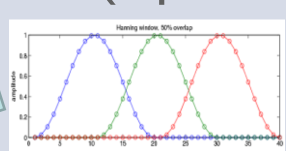


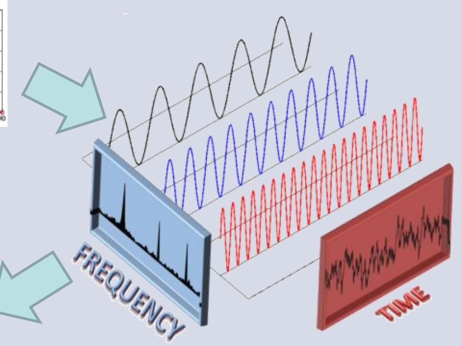


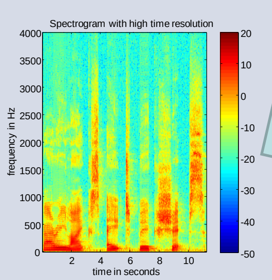
- Application of window function



Block-wise Frequency-Analysis (Spectrogram)







Time *vs.* Frequency Resolution

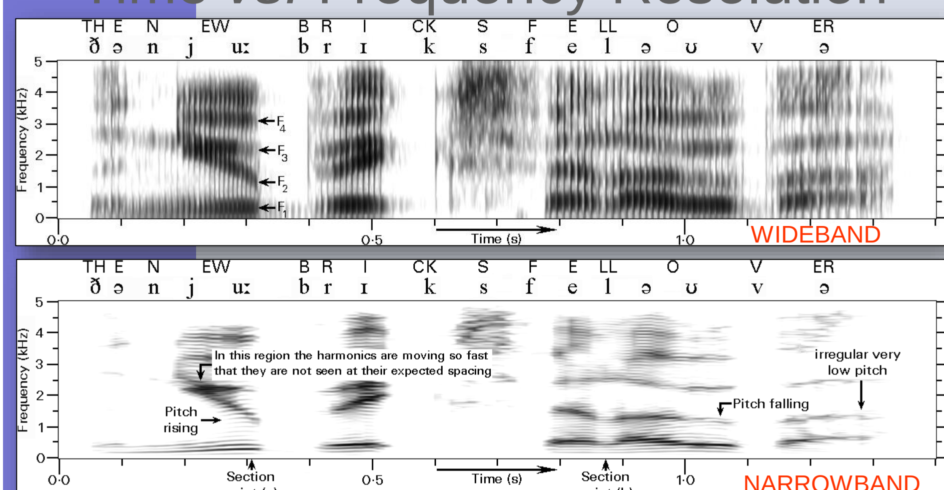
Frequency resolution = *1/NT (Hz)*

- Increasing the analysis frame N
  - decreases the spacing between the spectral components
  - reduces the ability to respond to changes in the signal
- Hence *large* N leads to 'narrowband analysis'
  - good spectral resolution
  - poor time resolution

- Decreasing the analysis frame N
  - increases the spacing between the spectral components
  - increases the ability to respond to changes in the signal
- Hence *small* N leads to 'wideband analysis'
  - good time resolution
  - poor spectral resolution

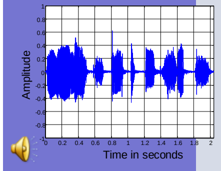
This is the *time-frequency trade-off*

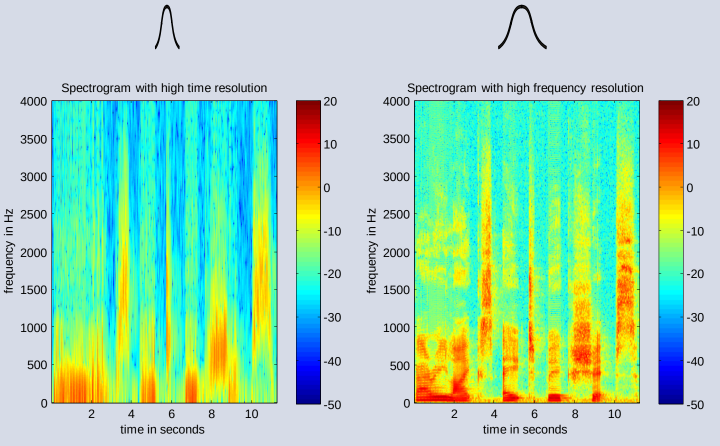
Time *vs.* Frequency Resolution



Taken from: Holmes, J. N., & Holmes, W. (2002). *Speech Synthesis and Recognition*: Taylor & Francis.

Time vs. Frequency Resolution of the Spectrogram





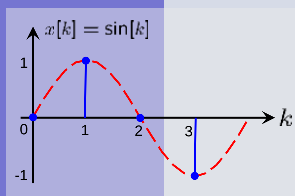
University of Sheffield

The Fast Fourier Transform (FFT)

- Implementation of the DFT requires the order of N^2 multiply-add operations
- By exploiting symmetry, it is possible to devise an algorithm that requires only $Nlog_2N$ multiply-add operations
  - e.g. for $N=2048$, the result is ~100x faster
- This more efficient algorithm is the … '**Fast Fourier Transform**' (*FFT*)
- The FFT requires that the window/frame should be a power of 2 in size
- This can be achieved by …
  - choosing the appropriate analysis frame size, and/or
  - zero-padding a frame to the nearest power of 2

University of Sheffield

DFT Example



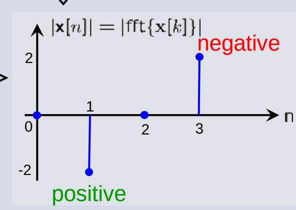
DFT: $\mathrm{x}[n] = \sum_{k=0}^{N-1} x[k]\, e^{-jkn2\pi/N}$

$$\Rightarrow \begin{cases} \mathbf{x[0]} = x[0]\cdot 1 + x[1]\cdot 1 + x[2]\cdot 1 + x[3]\cdot 1 = 0 \\ \mathbf{x[1]} = x[0]\cdot 1 - x[1]\cdot j - x[2]\cdot 1 + x[3]\cdot j = -2j \\ \mathbf{x[2]} = x[0]\cdot 1 - x[1]\cdot 1 + x[2]\cdot 1 - x[3]\cdot 1 = 0 \\ \mathbf{x[3]} = x[0]\cdot 1 + x[1]\cdot j - x[2]\cdot 1 - x[3]\cdot j = 2j \end{cases}$$

Discrete Fourier transform matrix:

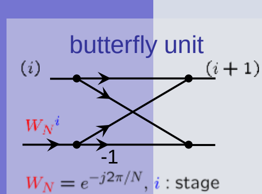
$$\mathbf{F} = \begin{bmatrix} 1 & 1 & 1 & 1 \\ 1 & -j & -1 & j \\ 1 & -1 & 1 & -1 \\ 1 & j & -1 & -j \end{bmatrix} \Rightarrow \mathbf{x}[n] = \mathbf{F} \cdot \mathbf{x}[k] \Rightarrow$$

column vector

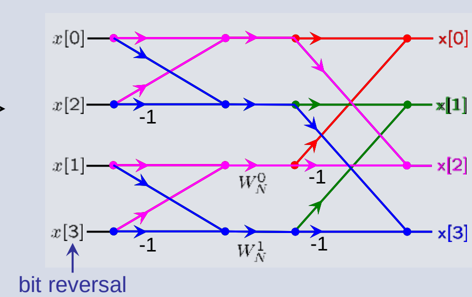


Matlab: `F = dftmtx(N)`

Fast Fourier Transform (FFT)



$\Rightarrow$



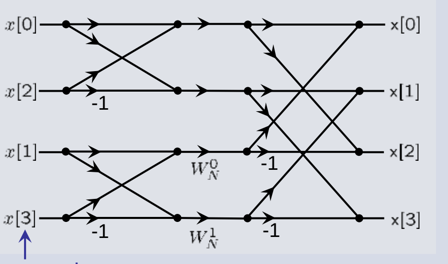
FFT is no different transform but an efficient algorithm for the DFT!

$$\mathbf{F} = \begin{bmatrix} 1 & 1 & 1 & 1 \\ 1 & -j & -1 & j \\ 1 & -1 & 1 & -1 \\ 1 & j & -1 & -j \end{bmatrix}$$

FFT (Bit Reversal)

index	binary	bitreverse	index
0	00	00	0
1	01	10	2
2	10	01	1
3	11	11	3

index	binary	bitreverse	index
0	000	000	0
1	001	100	4
2	010	010	2
3	011	110	6
4	100	001	1
5	101	101	5
6	110	011	3
7	111	111	7



bit reversal

Input samples (or output samples) have to be use in *bit-reverse* order!

This slide set has covered …

- Fourier Transform
- The discrete Fourier transform (*DFT*)
- Time versus frequency resolution
- Windowing
- The fast Fourier transform (*FFT*)

Any Questions ?

Raise during lecture or post in the Blackboard Discussion group

University of Sheffield

43

Next time …

Linear Filters

University of Sheffield

44